ChatGPT

Chintu Personal AI Assistant – Implementation Blueprint and Feasibility Study

Introduction and Vision

Chintu is envisioned as a **cross-device, privacy-first personal AI assistant** that operates seamlessly across a solo developer's ecosystem of devices (Windows laptop, macOS MacBook, Android Pixel 7 Pro, and iPhone XS). It aims for *"JARVIS-level"* intelligence [1] by leveraging on-device capabilities with optional cloud boosts. The assistant will handle a range of user-defined tasks – from coding assistance and file/app automation to job application management, personal knowledge recall, web research, and smart system control – through a **modular, multimodal interface**. Initial rollout focuses on desktop (Windows/macOS) with **text and voice** interaction, progressively adding **gesture and visual inputs** in later stages. Crucially, Chintu's architecture prioritizes **distributed computing** and **data privacy**: user data stays local by default (local LLMs and tools), while cloud APIs (OpenAI GPT-4o, Google Gemini, etc.) are invoked *only* with user permission for complex tasks. This feasibility study presents a comprehensive plan covering technical architecture, a 12-week development roadmap, tech stack choices, resource requirements, best practices, multi-device integration, and risk mitigation strategies.

Technical Architecture Overview

Chintu's system architecture is designed as a network of cooperating components across devices, ensuring resilience and efficiency. **Figure 1** outlines the key modules and data flow (from user input to action execution), described below:

- **Local LLM Core (Brain)**: At the heart is a local Large Language Model running via a distributed inference engine. We propose using **prima.cpp** – a state-of-the-art on-device distributed LLM system [2] – to harness compute from all personal devices. Prima.cpp (with its Halda scheduler) can partition a 30–70B parameter model across heterogeneous hardware (CPU/GPU mix, different OS) over Wi-Fi, enabling near real-time performance even on consumer devices [3]. This allows Chintu to run advanced models like a 70B Llama 3 without relying on cloud, achieving privacy and low latency. Each query is processed by the LLM **in a staged pipeline**: input arrives, is checked (for privacy/safety), then fed to the LLM which produces a response or plan.

- **Agent Orchestration Layer**: Rather than free-form agent conversations (which risk looping or derailment [4]), Chintu adopts a **deterministic state-machine orchestration** (codenamed *LangGraph*). The assistant's loop is modeled as a directed graph of states – e.g. **Listen** (wait for input) → **Privacy Check** → **Think** (LLM reasoning) → **Plan** (tool selection) → **Act** (execute tool) → **Learn** (store memory) → back to Listen [5]. This structured approach ensures each cycle follows a predictable sequence, improving reliability over unconstrained multi-agent chats. The orchestration logic can be implemented in Python using asynchronous loops or a framework like LangChain, but with custom control to enforce the state graph. High-level reasoning (multi-step plans) can be

managed by the LLM itself or by a specialized planner module, but always bounded by the state machine and *Constitutional AI* guardrails (to prevent unsafe actions). For complex tasks, **tool plugins** are invoked – such as code execution, web browsing, or file manipulation – with results fed back into the loop.

- **Privacy & Security Filter (Firewall)**: Before any user data is processed by the LLM or sent between devices, it passes through a *privacy filter pipeline*. We integrate **Microsoft Presidio** (regex/text filters) and **GLiNER** (advanced Named Entity Recognition) to scrub sensitive personal identifiers. Presidio provides deterministic redaction of common PII patterns (emails, credit cards, etc.) [6], while GLiNER (a lightweight transformer-based NER) can catch custom or context-dependent entities [7] with higher accuracy than small LLMs. This two-tier filter (running on-device) ensures that e.g. a phone number in a voice command or a screenshot never leaves the local network unmasked [8] [9]. The filter also acts as a **safety net** for content moderation (preventing the assistant from producing disallowed content). In practice, the Pixel phone can serve as the **"privacy gateway"** – capturing raw input (speech or text) and performing on-device PII masking before forwarding it to other nodes [10].

- **Long-Term Memory Store**: To provide continuity and learning over time, Chintu employs a **hybrid memory system**. We select **Zep (Graphiti)** as the long-term semantic memory backend [11] [12]. Zep represents knowledge as a *temporal knowledge graph* rather than unrelated vector embeddings – it stores entities and relations with time stamps, allowing the assistant to understand evolving facts (e.g. "User likes sushi" -> later "User is allergic to seafood" will update the graph instead of conflicting) [13] [14]. This is a step beyond simple vector databases; Zep's approach consistently outperforms standard RAG (Retrieve-And-Generate) in multi-hop reasoning benchmarks [15]. For immediate short-term memory (recent conversation context, session-specific preferences), a lightweight store (e.g. **Mem0**) is integrated for O(1) retrieval [16] [17]. The combination ensures that Chintu can recall long-term user information (projects, history, personal facts) while staying responsive in dialogue. Zep will be self-hosted (running on the primary node with a Postgres DB) to avoid external dependencies [18].

- **Multimodal Input Modules**: Chintu's interaction modalities are modular, allowing incremental addition:

- **Voice Input**: A speech-to-text (STT) engine processes user voice commands. We recommend using OpenAI's **Whisper** model (or its successors) fine-tuned for real-time use. By 2025, on-device ASR models like *Whisper v3 Turbo* or **Moonshine** offer 20-30× real-time transcription speeds with high accuracy [19], enabling low-latency voice control. The Pixel 7 Pro, with Google's on-device ASR capabilities, will initially act as the always-on "ear" for wake-word detection ("Hey Chintu"). It can run a lightweight wake-word model continuously and invoke Whisper/Moonshine for full transcription when triggered. The transcribed text then enters the agent pipeline. For voice **output**, a text-to-speech module (such as Coqui TTS or OS-native TTS) will vocalize responses when appropriate, creating a hands-free experience.
- **Gesture Input**: Later versions will interpret hand gestures via the camera as control commands. Using **MediaPipe** hand-tracking, Chintu can recognize gestures (like a certain sign or motion to trigger actions). The raw gesture data is stabilized with filters (e.g. a One Euro Filter) to avoid jitter [20] [21], ensuring smooth interpretation for tasks like moving the cursor or confirming a command. This *"action layer"* translates specific user gestures into system inputs – for example, a thumbs-up gesture might mean "proceed with suggested action". Gesture control will rely on computer vision

processed on the device with a camera (the laptop webcam or phone camera) and will be carefully calibrated for low false-positives (requiring explicit or unique gestures).

- **Visual Input (Vision)**: To truly act as a digital "eye," Chintu incorporates a vision module to understand screens and images. We will integrate **Microsoft OmniParser v2**, a cutting-edge screen parsing AI, which can convert arbitrary GUI screenshots into structured data (elements, icons, labels) that an LLM can reason over [22]. By *"tokenizing"* the pixels of the UI into text-like elements, OmniParser enables Chintu to locate buttons, menus, or links on screen and decide how to interact with them. This is critical for robust UI automation – it bridges the gap where DOM-based approaches fail (e.g. non-HTML interfaces). OmniParser v2 significantly improved detection of small UI elements and runs 60% faster than the previous version [23], making real-time screen analysis feasible. Combined with a multimodal LLM (like Qwen-2.5 VL or GPT-4o's vision mode), Chintu can achieve state-of-the-art accuracy (39.6% on the ScreenSpot benchmark vs <1% without OmniParser) in identifying the correct on-screen targets for actions [24]. In practice, a **Screen Capture tool** on the PC will feed screenshots to OmniParser, and the structured output will allow the agent to click the right UI element via OS APIs. This gives Chintu "eyes on the screen" to complement its "brain".

- **Text & Chat Interface**: Of course, textual interaction remains fundamental. Chintu provides a console or chat window on desktop for users to type commands or chat. This interface is always available and doubles as a debug view, showing transcripts of voice interactions or the reasoning steps when needed (especially useful during development and for transparency). The text interface is essentially the default fallback if voice/gesture aren't used.

- **Task Automation & Tools**: Much of Chintu's power comes from its tool integration for acting on the user's behalf:

- *Coding Assistance*: Chintu can function as a coding copilot, answering programming questions, generating code snippets, or even coding autonomously. It will utilize a code-specialized model or mode (e.g. a local Code LLM like Code Llama or GPT-4o's coding capabilities) when the user enters a programming query. It can also interface with the developer's environment – for example, through an API or plugin to the code editor/IDE, allowing it to fetch context (open files, error logs) or apply changes (with user approval). Over time, this could evolve into an **OpenHands "Software Engineer" agent**, where Chintu can create or modify files, run tests, and debug within a sandbox. We plan to integrate **OpenHands Agent SDK** for this purpose, as it provides a framework for AI agents to carry out coding tasks locally with proper tooling. OpenHands supports *distributed agent deployments*, meaning the agent logic can run on the user's machine while heavy tools (compiling, browser automation) run in sandboxed containers [25] [26]. Using this, Chintu could, for example, generate a Python script and execute it in an isolated Docker container (to prevent harming the host system), achieving automation safely.
- *File and App Automation*: Chintu can perform OS-level actions like opening applications, editing files, moving data between apps, or adjusting system settings. To ensure safety, these operations go through a validation layer – e.g. "delete file" commands will prompt for confirmation and check against protected directories [27]. In early stages, simple automation can be done via scripting (Python **PyAutoGUI** for GUI interactions, shell commands for file operations, AppleScript/PowerShell for system settings). However, these can be brittle, so the plan is to transition to more robust methods: for browsers and web apps, using **Playwright/Selenium** automation within a contained environment [27] (or even leveraging the **OmniParser+LLM** approach for non-DOM interfaces); for native apps, using OS-specific automation APIs (Accessibility APIs on macOS/Windows) or the vision-

based click approach. All such actions are logged and reversible where possible (undo or backups) to maintain trust.

- *Job Application Agent*: One of the unique user-defined tasks is automating job applications. Chintu will include a specialized workflow for this, as outlined in the Chintu v2.2 PRD. It will parse the user's resume, tailor it to match a job description, auto-fill web forms and track applications [28] [29]. For example, given a target job posting, the assistant can extract key required skills and modify the user's resume/cover letter to highlight those (using LLM text generation). It then uses browser automation to submit the application on sites like LinkedIn or Indeed [30], interfacing either via public APIs or simulated form fills. Each application submission is done with user approval and logged (with status updates in the personal database). This agent will reuse components: it uses the web automation tool for form filling, the memory store to avoid re-applying to the same job twice, and the LLM for generating cover letters.
- *Personal Knowledge Management*: Chintu will serve as an extended memory for the user – recalling personal notes, past conversations, or important info on command. The **long-term memory (Zep)** plays a key role here, as it can be queried for facts like "What did I tell you about project X last month?" and provide context to the LLM for a detailed answer. We will also implement a **Personal Data Parser** (using OmniParser and/or direct file parsing) to ingest the user's files or emails (with consent) into the memory store, enabling powerful search and recall. For instance, the user can ask "Summarize the key points from the PDF I opened yesterday," and Chintu can retrieve the PDF text and summarize it. This essentially becomes the user's second brain, while respecting privacy (data never leaves the devices).
- *Web Research and Browsing*: For queries requiring up-to-date information or browsing (e.g. "Find me recent news on AI regulations" or "Compare the prices of laptops"), Chintu can perform web research. A headless browser or search API will be used via an *Internet Tool*. This tool, when invoked, conducts a search (e.g. through Bing Web API or Google if accessible) and scrapes relevant pages. The results are then summarized by the LLM and presented, with citations if needed. Using OpenHands or similar, this browsing can be done in a sandbox container to prevent any malicious web content from affecting the host. Initially, simply opening the system's browser in an automated way with Playwright to grab content might suffice [27], but as a more integrated approach, the assistant could have a text-based browser mode. **Note**: Web access will be disabled by default or clearly indicated to the user, to ensure no inadvertent data leakage (the user must request a web search explicitly).
- *Smart Device & System Control*: Chintu can also control IoT or system features. For example, it might interface with smart home APIs (turning lights on/off via a REST call to a local smart hub) or change phone settings (set Do Not Disturb, navigation to a location, etc.). This requires integration with device-specific automation hooks. On Android, **ADB (Android Debug Bridge)** can be used in Wi-Fi mode to send intents or simulate taps on the Pixel [31]. On iPhone, **Siri Shortcuts** automation can be leveraged: we can set up personal shortcuts that perform actions (like "send message to X" or "open app Y") and expose them via URL schemes or a simple HTTP webhook. Chintu's controller on PC can hit these webhooks to trigger the shortcut on the iPhone [32]. This allows a form of cross-device RPA (Robotic Process Automation) where the assistant orchestrates tasks on whatever device is needed – e.g. if a text message needs to be sent from the iPhone, the PC instructs the iPhone's shortcut to do so.

All these components communicate over a **secure local network**. Using **Tailscale** (a mesh VPN), the devices form a private, encrypted overlay network for Chintu's inter-process communication [33]. Each device runs a lightweight **agent service** that registers its capabilities (compute resources, tools it can execute, etc.). For example, the Pixel's agent registers a "speech input + privacy filter" service, the Dell laptop registers "LLM

inference and vision processing" service. A simple **service registry** or discovery protocol (even something as simple as static IP addresses or mDNS hostnames given Tailscale) will allow the orchestrator to route requests to the appropriate node. Communication can use gRPC or WebSocket-based RPC calls for efficiency and ease of multi-language support. All messages and payloads (like audio chunks, screenshots, or text) are encrypted in transit (Tailscale provides WireGuard-level encryption by default [34]).

In summary, Chintu's architecture marries **local AI models**, **knowledge graphs**, and **an array of automation tools** in a distributed fashion. Each module is chosen for **feasibility and performance**: local inference for speed/privacy, small specialized models (GLiNER, Whisper, etc.) for edge tasks, and only optional use of cloud AI (like OpenAI's GPT-4o or Google's Gemini) as a fallback for extremely complex queries. Notably, OpenAI's **GPT-4o** ("omni" multimodal GPT-4) could be tapped for its advanced reasoning and vision integration [35], and Google's **Gemini 2.5 Pro** can be leveraged for state-of-the-art code and reasoning tasks [36] – but these will be called through secure APIs *only if* the local model's response is insufficient and the user has opted in to cloud help. This hybrid strategy ensures Chintu is both **powerful and privacy-preserving**.

Technology Stack and Implementation Choices

Building Chintu as a solo developer requires leveraging high-level frameworks and existing libraries to accelerate development, while ensuring cross-platform compatibility:

- **Programming Languages**: The core orchestration and logic will be implemented in **Python** (due to its rich AI ecosystem and faster development cycle). Python allows easy use of PyTorch (for model integration), LangChain/LangGraph for agent logic, and numerous libraries (Presidio SDK, GLiNER, etc.) that are either Python-based or have Python bindings. Performance-critical portions (like model inference) are offloaded to optimized binaries (e.g. llama.cpp/prima.cpp in C++). For certain mobile or UI components, other languages will be used: Swift/Objective-C for iOS integration (Shortcuts or app, if needed), and Java/Kotlin for any Android service (though we can attempt to run Python on Android via **Termux** or Chaquopy for quick prototyping). However, using *Python across devices* is feasible on desktops and Android (with Termux or an embedded interpreter), but not straightforward on iOS (where a dedicated Swift app would be needed). To mitigate this, the architecture keeps Python mainly on the PC side and uses simple HTTP interfaces for mobile actions, so mobile apps can be thin clients in native code.

- **LLM Inference Engine**: We will use **Llama.cpp / prima.cpp** for running local models. On devices with sufficient RAM, models like Llama 3 (the hypothetical next-gen Llama) or smaller variants can be loaded in 4-bit quantized form. **Ollama** can be utilized especially on macOS: it provides a convenient way to download and run local models via a CLI/API [37] (and supports both macOS and Windows). For distributed mode, prima.cpp's runtime (which is a fork/extension of llama.cpp) will be set up on each capable device (Dell and Mac primarily) and the Halda scheduler configured to balance the load [38]. The Dell (Windows) might run a WSL2 or Linux container if needed to facilitate compatibility of prima.cpp (or we compile it for Windows). The Mac's M1 will run the ARM build. The Pixel can run a lighter instance possibly via Termux (compiling prima.cpp for ARM64 Android) [39] – allowing it to host parts of the model (e.g. less memory-intensive layers). If distributing the model proves too complex initially, a fallback is to run the full model on the Dell (which has the most resources) and simply use the other devices for other tasks (voice, etc.), adding distribution later.

- **Frameworks & Libraries**:

- *Agent Orchestration*: **LangChain** (Python) or a similar framework will be employed for managing prompts, tools, and conversational memory. However, to implement the custom state-machine approach, we might use LangChain's lower-level components and maintain state explicitly. Another emerging option is **LangGraph** (if available as described in architecture docs) which treats agent steps as graph nodes – if an open-source version exists, that could fit well. Otherwise, a bespoke implementation using Python state management (asyncio loops and event handlers for each state) will be done.
- *Memory and Database*: **Zep** will be used for the knowledge graph memory. If Zep provides a self-hosted server (with Graphiti), we will deploy that (likely via Docker on the primary node). If not, a custom lightweight graph DB could be built (networkx or Neo4j) – but Zep's design is preferred for its optimizations. For quick key-value storage (Mem0), a simple Redis or even in-memory Python dict with persistence could suffice. The data (knowledge graph, etc.) will be stored on disk on the primary machine; backups can be enabled (optionally to an encrypted cloud store or external drive) to prevent loss.
- *Networking*: **Tailscale** for networking as mentioned. For messaging, **gRPC** is a strong candidate – define a proto with services like `SendText(input)`, `RequestToolExecution(tool, params)`, etc., and generate stubs for Python, Swift, etc. Alternatively, a REST API with HTTP+JSON could be simpler to debug. Given the small scale (4 devices), simplicity might trump performance, so an HTTP+WebSocket approach could work: e.g. the Pixel app opens a WebSocket to the PC for streaming audio/text and receives responses.
- *User Interface*: Cross-platform UI is challenging but important. After evaluating frameworks in a separate UI/UX analysis, **Flutter** emerges as a top choice for a unified UI layer. Flutter's **Skia/Impeller engine** can render real-time audio waveforms at 60+ FPS smoothly [40] [41], which is useful for visualizing the assistant's "listening" state or voice responses. It also supports desktop (Windows, macOS, Linux) and mobile from a single codebase. We will consider building a Flutter UI that runs on each platform, providing a **consistent experience** – e.g. a small overlay window with a microphone icon and chat display. One challenge noted is macOS window transparency and click-through for overlays [42], which might require some native code workarounds. If Flutter development proves too time-intensive for the solo developer initially, a simpler UI will be used for MVP: possibly a **Tauri** app (Rust backend, WebView frontend) for desktop and a minimal native app for mobile. Tauri is lightweight and uses system WebView (small footprint) [43], but it may have a bit more latency for high-frequency updates due to its JS bridge [44]. Weighing options, the plan is:
    - Phase 1: Minimal UI – perhaps a Python Tkinter or Electron window to show text I/O and a button to trigger voice – to get things working.
    - Phase 2: Flutter-based UI – once the core is stable, invest time to create a polished Flutter interface across devices, with features like a *floating overlay mode* (picture-in-picture style window) that can persist on screen and be summoned by voice [45].

- *Mobile Integration*: On Android, we can develop a small **Android Service** (in Kotlin) to handle the microphone and hotword detection (utilizing Google's on-device ASR or an open model). This service would connect to the cluster via Tailscale IP to send transcripts and receive commands (which it executes via intents or UI interactions). On iOS, due to background restrictions, a **Shortcut-based approach** might be used: the user can say "Hey Siri, launch Chintu" to start the app or listening, but continuous background listening isn't allowed on iOS without the app in foreground. We might need the user to open the iOS app to use voice on iPhone. The iOS app (SwiftUI or Flutter if possible on

iOS) will primarily act as a client to display answers and send requests; any deeper integration (like toggling settings) will go through Shortcuts automation since Apple sandboxes apps heavily.

- **Models & AI Services**:

- Core LLM: **Llama 3.2** (hypothetical future version of Llama) or a similar local model ~70B parameters for general intelligence. If Llama 3 is not available, Llama 2 70B or Code Llama 34B can be used as a stand-in. These will be quantized to fit on hardware; multi-device inference via prima.cpp ensures no single device must hold the entire model in RAM [46]. For specialized needs: a smaller 2-7B model can be kept for *speculative decoding* to speed up responses (drafting tokens while the big model works, as suggested in v3 plan) [47], and possibly a local code model for programming queries.
- Speech: **Whisper** (open-source ASR) or **Coqui STT** for speech-to-text; **Coqui TTS** or Apple's AVSpeechSynthesizer for text-to-speech output on Mac/iOS (for natural voice responses).
- NER/Filtering: **Presidio** (Python library by Microsoft) for regex and pattern matching (e.g. we'll configure it with policies to detect phone numbers, addresses, etc.), and **GLiNER** model (available via HuggingFace or PyPI) for context-aware NER. GLiNER is compact (couple hundred MB at most) and can run on CPU with <100ms latency [48], suitable for real-time filtering.
- Vision: **OmniParser v2** (likely available via Microsoft's GitHub or Azure AI) running on Windows (Dell). It requires a machine with a GPU for acceptable performance [49] – we plan to use the Dell's NVIDIA GPU for this. If GPU is not available, OmniParser might be too slow; in that case, fallback is partial automation with template matching or OCR, but the goal is to include a capable GPU in hardware planning.
- Agent Tools: **OpenHands SDK** (Python package) for agent actions, especially coding tasks. Also possibly leverage **E2B (Enterprise to Bot)** platform for sandboxing code execution – E2B offers an API to run code in cloud sandboxes securely [50] [51]. A free tier is available for low usage, which a single dev can likely stay within [52]. For browser control, **Playwright** in a Docker container (to isolate any browser automation from the host).

- Optional Cloud APIs: **OpenAI GPT-4o** (which is GPT-4 Omni, capable of text, image, audio input) [35] and **Google Gemini Pro (2.5 or 3)** via Vertex AI [36]. These will be integrated through their SDKs (OpenAI Python API, Google Cloud client). Usage will be minimal and only triggered when the user explicitly asks for extra analysis or if the local model yields an uncertain response and user permits a second opinion.

- **Deployment Strategy**:

- During development, we will run the components on the developer's devices directly (e.g. Python scripts on each). For distribution, packaging will be needed. We intend to containerize certain services for consistency: e.g. run Zep, OmniParser, and the LLM orchestrator in Docker containers on the host machine(s). This ensures easy startup and isolation. Tailscale can run as a service on each device to maintain connectivity.
- For the user-facing app (Chintu UI), we will package it for each platform: a Windows .exe or MSI installer (if Flutter, using `flutter build windows`), a macOS .app, an Android APK, and an iOS app via TestFlight for testing (since App Store distribution might not be initial goal). These apps will contain the UI and a client that communicates with the core services. On desktop, the UI app and core could run on the same machine; on mobile, the app will mostly be a client to the core on desktop.

- The assistant should ideally auto-start on the main devices (e.g. launch at login on laptop in background) so that it's always available. A small background service with just the hotword listener on Pixel will start on boot (Android allows that with appropriate permission). The iPhone, being limited, might not have always-on processes; the user would open the app when needed.

Feature Breakdown and Development Timeline (12 Weeks)

The following timeline outlines a feasible 12-week plan to develop Chintu's features incrementally. Each week's focus builds on previous work, adding one capability at a time and ensuring integration and testing:

Week 1: Infrastructure Setup & Core Framework
- **Task**: Set up the foundational infrastructure for a distributed assistant.
- **Key Actions**:
- Establish secure device networking with Tailscale (create a private mesh VPN linking the Dell, MacBook, Pixel, and iPhone) [33] . Verify all devices can reach each other by hostname/IP.
- Initialize a Git repository and coding environment with AI-assisted coding tools (the developer can use GitHub Copilot or similar to speed up iterations).
- Configure a basic **agent loop** in Python on the primary PC: a simple REPL that reads text input (from console) and responds using a placeholder stub (e.g. echoing or a trivial hard-coded LLM response). This is to lay the groundwork for the state-machine loop.
- Research and choose libraries for each module (confirm PyPI packages for Presidio, GLiNER, Zep client, etc. are available).
- **Deliverable**: *Hello World* of the architecture – the devices are networked, and you can send a dummy message from one to another (e.g. Pixel to PC) and get a response. Documentation of the environment (ensuring Python runs on Windows and Mac, maybe Termux on Android prepared).

Week 2: Local LLM Integration (Text MVP)
- **Task**: Get a local LLM running and connect it to the agent loop for basic Q&A over text.
- **Key Actions**:
- Install and configure **Ollama/llama.cpp** on the PC for a chosen model (e.g. Llama-2 13B as a starting point due to resource constraints). Test generating answers to prompts locally [53] .
- Implement the `Think` state of the agent: user text -> LLM prompt -> LLM completion -> return answer. At first, use a single device (Dell) for inference without distribution. Ensure the pipeline works end-to-end in a simple manner (type a question in console, get an answer from the local model).
- If using Ollama, use its API to run the model; otherwise integrate llama.cpp through a Python binding (like `ctransformers` or `llama-cpp-python`). Optimize for CPU inference (quantize model if needed to fit in RAM).
- Basic prompt template: Include a system prompt establishing the assistant's identity and any basic instructions (e.g. to be helpful and use tools if asked). This will evolve, but set a foundation.
- Start a simple logging mechanism for each conversation turn (for debugging and future memory).
- **Deliverable**: A rudimentary text-based chatbot running **locally** on the desktop. Demonstrated by asking a factual question and getting a coherent answer. This proves the core LLM integration feasibility.

Week 3: Privacy Filter & Guardrails
- **Task**: Integrate the privacy and safety filtering stage before the LLM processes input or produces output.
- **Key Actions**:
- Set up **Microsoft Presidio** in Python and define a policy for PII (e.g. detect emails, phone numbers,

names). Test it on sample text (e.g. “Call John at 555-1234” -> ensure it would mask the number).
- Integrate **GLiNER** model for NER: load a pre-trained GLiNER (e.g. `gliner_large-v2`) and test custom entity extraction [7]. Define entity labels for things we consider sensitive (PERSON, ORG, etc., plus custom like “Credential” for passwords). This can catch context Presidio might miss.
- Modify the agent loop: after `Listen` (or receiving input), run the input through Presidio & GLiNER. Remove or mask sensitive info (the system might replace them with [MASK] tokens or abstract them). Do similar filtering on the LLM’s output before delivering to user (to prevent accidental leaks).
- Implement basic *content moderation* rules as well (e.g. block obviously harmful requests). This could use OpenAI’s content filter if allowed locally or a simple keyword list to start.
- Begin adding **Constitutional AI** prompts: define a set of fundamental “do no harm” instructions always prepended to the LLM (like “Never reveal personal info. Always ask for confirmation before destructive actions,” etc.). These act as built-in safety rails from the outset.
- **Deliverable**: Enhanced pipeline where if user input contains something like an email, the system will sanitize it *before* the LLM sees it, and similarly sanitize the output. We can demonstrate by inputting “My credit card is 4111-xxxx, what’s my number?” and the assistant should refuse or mask it, showing the privacy filter in action [54]. This week ensures **privacy-first design** is enforced early.

Week 4: Distributed Compute – Multi-Device LLM & Networking
- **Task**: Expand LLM inference to use multiple devices and set up inter-device communication APIs.
- **Key Actions**:
- **Prima.cpp setup**: Compile or install prima.cpp on the Dell and MacBook (Windows might require WSL as fallback). Use a suitable model (maybe Llama2 30B) and configure the Halda scheduler. Run benchmarks to see if splitting across the two devices yields speed improvements. If the Pixel can be included (via Termux), attempt to include it as well, perhaps holding the KV cache or smaller parts [55].
- If prima.cpp is complex to integrate directly, consider using its results: for blueprint, we assume success; but in development, if blocked, document a fallback to run half model on each device manually (e.g. using model parallel over socket – not trivial, so likely rely on prima which handles it).
- Develop a basic **RPC communication** between devices: for instance, create a small Flask or FastAPI server on the PC to expose endpoints like `/ask` that the Pixel could call with audio or text. Conversely, an endpoint on Pixel to trigger TTS or phone actions. Use the Tailscale IPs for addressing. Ensure connectivity and basic auth (even though within VPN, maybe use an API token to avoid any misuse).
- Test a scenario: Pixel sends a request to PC’s API (“What’s the weather?”), PC processes with LLM, PC sends back answer, Pixel receives and (for now just logs it). This is laying ground for real voice integration.
- Also consider **cloud fallback hook**: set up the OpenAI and Google API credentials this week. Write a function `ask_cloud(question)` that can query GPT-4o or Gemini for future use. Not integrated yet, but ready for manual testing.
- **Deliverable**: A **distributed inference demo** – e.g., run the assistant with the model split on two machines and show that each device’s utilization goes up and response time goes down, indicating successful load sharing (if possible). Also a working basic API layer among devices. At this point, Chintu can still be interacted via text, but now the groundwork for multi-device operation and eventual voice UI is prepared [46].

Week 5: Memory Integration (Long-term & Session)
- **Task**: Give the assistant a memory. Integrate the Zep graph database and short-term context store.
- **Key Actions**:
- Deploy **Zep (Graphiti)**: Run Zep’s open-source server (likely via Docker) on the primary node [56]. Create a Postgres instance (Docker or local) to back it. Ensure it’s accessible (probably just on localhost or within

Tailscale for other nodes).
- Implement an **Episodic Memory Extractor**: after each conversation or task, summarize key facts or decisions and send to Zep as knowledge updates. For example, user says "I started a new project on machine learning," the assistant will store `[timestamp]: User has a new ML project`. Use Zep's API to upsert nodes/edges (like Person "User" — ownsProject -> "ML project"). This may involve using Zep's SDK or directly sending data via its REST endpoints.
- Integrate **Mem0** for quick context: perhaps use an in-memory Python dict or Redis to store the last N user instructions or preferences. On each new query, the agent can look up relevant recent items (e.g. "user preferred language=Python") and include that as context or prompt.
- Modify the LLM prompt construction: Now, before calling the LLM in `Think`, query Zep for any relevant knowledge (Zep might have a function to query related info given the current conversation). Include that info in the prompt (as context paragraphs or a structured "Knowledge:" section). This will allow the assistant to recall things from past sessions.
- Test memory: Have a conversation in which the user provides some info, then later ask about it. E.g. *User:* "My favorite food is sushi." (store in memory) … later *User:* "What's my favorite food?" The assistant should fetch from Zep that the user likes sushi (but also note if there was a change). If possible, simulate a change like "Actually I hate sushi now" and see if Zep's temporal aspect can handle updating the relationship [13] [14].
- Ensure data in Zep is encrypted or at least in a secure environment (since personal data is now being stored). If not fully confident, restrict memory to non-sensitive info for now (given privacy – though we filter input, presumably memory stores the filtered version or a safe representation).
- **Deliverable**: The assistant now has **persistent memory**. Demonstrated by the assistant remembering a fact from one query to a later one (even after a restart, if we persist the DB). This establishes that Chintu can learn user context over time, hitting the requirement for personal memory.

Week 6: Agent Skills – Tool Use and Planning
- **Task**: Expand the assistant's capabilities by enabling tool usage (coding, web search, automation actions) through the orchestrator.
- **Key Actions**:
- Integrate a **tool plugin system** in the agent loop. For example, use LangChain's tool abstraction: define tools like `search_web(query)`, `open_url(url)`, `run_code(code)`, `send_email(recipient, content)`, etc., each with a function implemented. The LLM can output a format (like a *thought* and *action* command) that the orchestrator parses and executes.
- Start with a simple tool: a **Web Search** tool. Use Bing's search API (or Google's if available) to implement `search_web`. Then a `summarize_page` tool that fetches a URL content and summarizes (you can utilize the LLM itself for summarization, or an external API). Test this: ask the assistant "What's the latest on AI regulations?" and see it use the tool (trace in logs) then give an answer with citations.
- Implement a **Code Execution** tool in a safe manner. Perhaps leverage something like the Python `exec` in a restricted namespace or, better, use OpenHands' approach with a sandbox. For now, an easy way: use the `subprocess` module to run a separate script or command, but ensure it's something trivial and safe (like math calculations, not system destructive). The goal is that the LLM can say: Action: `run_code` with content, and we execute it and return output. This is a stepping stone to more complex coding tasks.
- Introduce the **planning** ability for multi-step tasks. This could be as simple as enabling the LLM to loop through thought->action->observation cycles. Ensure the state-machine allows iterative planning: e.g. state `Think` may output multiple steps (or have sub-states for planning). Possibly integrate an existing agent loop implementation (LangChain's AgentExecutor or an open-source agent like BabyAGI style). We should be cautious to avoid infinite loops – set a reasonable cap on steps and have fallback to ask user if unsure.

- Initial support for **file operations**: add a tool like `read_file(path)` and `write_file(path, content)` with safeguards [27]. This allows the assistant to retrieve data from local files or create a text file if needed (e.g. drafting an email). Use case: user says "Draft a cover letter for job X" – the assistant could call `read_file(resume.txt)` to get user's resume from a known location, then compose a letter referencing it. These are preparatory steps for job applications and coding help.
- By end of week, the assistant should have a handful of tools and be able to decide when to use them. We will manually test scenarios: math calculation (should use run_code or an internal calculator tool), factual question (maybe use web_search), request to open an app or file (the assistant suggests an action, though we might not fully wire it to OS yet, just simulate).
- **Deliverable**: A **tool-using AI assistant**. In logs, we can see something like: *Thought: "I need to find information online." Action: search_web["OpenAI latest news"]*... then it gets results and formulates an answer. This demonstrates the shift from a pure Q&A bot to an **action-oriented agent**.

Week 7: Voice Interface Integration
- **Task**: Add voice input and output capability on top of the text chat pipeline.
- **Key Actions**:
- Incorporate **Speech-to-Text** using Whisper (small model for now). Install `whisper.cpp` or use OpenAI's Whisper API as a temporary measure (though cloud, it can be a stopgap until Moonshine local is integrated). Ideally, use a local model to keep privacy. Optimize for real-time: possibly use VAD (voice activity detection) to know when user finishes speaking.
- On the Pixel 7, implement the **hotword detection** ("Hey Chintu"). Android's `VoiceInteractionService` or Google's Hotword API could be used if accessible; otherwise use a lightweight model like Porcupine or Silero VAD + keyword spotting. Ensure it runs in background with minimal battery. When hotword is detected, start capturing audio and stream or send it to the PC for transcription (or transcribe on Pixel if model is small enough).
- On PC side, create a **Microphone thread** (if user chooses to use PC mic instead of phone) – possibly allow both triggers. But to keep it simple, consider Pixel the primary voice input device for now.
- Connect the STT: once speech is transcribed to text, feed it into the existing agent pipeline as if it was typed. Then capture the response.
- Add **Text-to-Speech** for responses: on Windows, could use the built-in SAPI voices; on Android, use Android TTS engine to speak; on Mac, use AVFoundation. For cross-platform ease, maybe integrate an offline TTS library (like Coqui) on PC and use Android's native on Pixel. The voice should clearly indicate the assistant's responses.
- Address UI/UX: Provide a chime or visual cue when hotword is detected (e.g. Pixel vibrates or the desktop overlay shows listening waveform). After answer is spoken, ensure it doesn't re-trigger hotword from its own voice – might need a brief mute.
- Test the **end-to-end voice query**: e.g. say "Hey Chintu, what's the weather tomorrow?" – Pixel detects wake, records, transcribes, sends to PC, PC uses a dummy tool (or real web API) to get weather, formulates answer, and then the Pixel (or PC) speaks out "Tomorrow it will be sunny and 75 degrees." Verify latency and adjust (maybe compress audio or use streaming decode to start thinking before full utterance ends).
- **Deliverable**: **Voice-enabled assistant**. This is a major milestone: the user can talk to Chintu via voice on at least one device and get a spoken response. It transforms the assistant into a truly conversational agent. We'll also document the achieved latency (aiming for under 2 seconds for short queries) and identify any issues (like background noise handling).

Week 8: End-to-End Integration & Refinement
- **Task**: This week is about hardening what's built so far – conducting integration tests, fixing bugs, and

optimizing performance.
- **Key Actions**:
- Perform **full system tests** of scenarios: e.g. “Chintu, open my VSCode and create a new Python file.” This would involve voice -> text, LLM deciding to do an automation action, and executing it. We might simulate parts if not fully ready (like just printing an intention rather than actually opening VSCode). Identify points of failure (did the STT mis-transcribe? Did the LLM hallucinate a tool name? etc.). Improve prompt or add constraints as needed.
- Optimize **latency**: introduce *speculative decoding* as mentioned – use a small local model to generate a few next tokens while the big model is processing, to make it feel faster. Also, ensure that the network overhead for distribution is minimized (maybe persist connections rather than open each time). If some steps are slow (e.g. Zep queries), consider caching frequent results.
- **Stability**: make the system robust to disconnects – e.g. if Pixel drops off network, the PC should handle that gracefully (maybe queue any messages or alert the user). Similarly, implement simple retry logic for RPC calls.
- UI improvements: if not yet, at least set up a basic **desktop UI** window that shows the conversation and a “Listening…” indicator. This could be a minimal Flask web interface or a Python GUI for now, unless Flutter/Tauri is already started. The user should have a way to see what Chintu heard and its response (important for trust and corrections).
- Security review: ensure that any tool that executes code or accesses files has checks. For example, sandbox the code execution (use `docker run` for OpenHands if possible by now, or restrict Python’s builtins if just exec). Also double-check that sensitive data (like any API keys in prompts) are not being logged or exposed.
- **Deliverable**: A **beta version** of Chintu running across devices reliably. By end of Week 8, a user could realistically use Chintu for simple tasks (ask a fact, set a reminder – which we could implement via writing to a file or calendar if time, small things) and the system would perform end-to-end. Documentation of how to run each component is updated, paving the way for adding the final advanced features.

Week 9: “Hands” – Coding Agent Integration (OpenHands)
- **Task**: Enable Chintu to perform advanced coding and automation tasks autonomously using the OpenHands SDK and sandboxing.
- **Key Actions**:
- Incorporate **OpenHands**: install the OpenHands Agent SDK (if open-sourced by now) into the project. Start a simple OpenHands agent locally that can do coding tasks. For example, the agent could create and edit files related to a user’s request.
- Use **DockerWorkspace** from OpenHands for isolated execution [57] [58]. Set up Docker on the primary machine if not already, and ensure we can launch a container to run code that the agent writes (this protects the host).
- Define a scenario: perhaps “Chintu, create a simple website that says hello.” The LLM portion might hand this to OpenHands which then generates an HTML file and a simple HTTP server code, builds it, and could even run it in the sandbox. We check that the code runs correctly in container.
- Leverage OpenHands’ **event streaming**: the SDK likely allows monitoring agent steps [59]. We can use this to keep the user informed (e.g. “Creating file hello.py… Running tests…”).
- Expand **toolset** with advanced actions: incorporate the automation of applications. For instance, try a Playwright automation within the Docker if possible (OpenHands might have built-in support for browser tasks). Another example: use OS-specific commands, like an AppleScript to send an iMessage (but only if in a safe list of actions).
- *Self-improvement*: begin exploring letting the agent modify its own skills. This is tricky, but OpenHands plus our memory could allow a form of self-extension – e.g. the assistant notices it lacks a tool, it could write new

code to add a tool. We won't implement full autonomous self-modification yet (due to safety), but we'll design the plugin architecture such that new Python modules can be hot-loaded if needed (with user confirmation).
- **Deliverable**: Chintu can act as a **basic software agent**, writing and running code on demand. A demonstration might be the assistant solving a coding challenge by writing a script and executing it to get the answer. This shows progress toward the "auto-dev" capability and gives Chintu real "hands" in the digital world.

Week 10: "Eyes" – Vision and Screen Understanding
- **Task**: Integrate the vision pipeline for GUI understanding using OmniParser, and possibly initial gesture recognition.
- **Key Actions**:
- Set up **OmniParser v2** on the Dell (with GPU). Follow Microsoft's instructions to get the model and any dependencies (it might use ONNX or PyTorch with CUDA). Use a sample screenshot (maybe take one of a calculator app or a settings window) and run it through OmniParser to get a JSON of UI elements. Verify that we can parse out text labels and coordinates of buttons [22].
- Create a **Screen Capture** function on both PC and Mac. On Windows, this could be done with something like MSS or PIL to grab the screen; on Mac, use Quartz APIs or a cross-platform tool. We need to capture either the full desktop or specific window content as needed. Possibly integrate a shortcut phrase to "what's on my screen?" to test capturing and describing via LLM.
- Use the LLM + vision model to implement **Visual Reasoning**: feed OmniParser's structured output and ask the LLM to interpret it. For instance, if a browser window is open with certain icons (back, forward, address bar), and user says "Refresh the page", the assistant should identify the "refresh" icon from parsed elements and then call a click action on its coordinates. This requires mapping parsed UI elements to automation actions. We'll likely implement a helper that, given an element (by text or type), uses an OS automation library to simulate a click or keypress at that element's location.
- If available, incorporate **Qwen-2.5-VL** or GPT-4V as an analyzer for images to cross-verify OmniParser's output (this might be beyond a solo dev's scope due to needing the model and compute, but conceptually, mention that the multimodal LLM can assist in reasoning).
- **Gesture Recognition**: As part of vision, if time permits, start implementing MediaPipe Hand tracking on a webcam feed. Recognize at least a couple of gestures (like open palm = move cursor mode, pinch = click). This might require significant tweaking, so if it threatens timeline, plan to do a basic demo only (not fully robust integration yet). Possibly defer full gesture control to post-12-week, but have groundwork done now (like the pipeline to get coordinates of hand and move mouse pointer via OS calls).
- **Deliverable**: Chintu gains **visual perception**. A test scenario: have a calculator app open and ask via voice "Chintu, press 1 2 3 and then plus on the calculator." The assistant should parse the screen, identify the "1", "2", "3", "+" buttons and simulate clicks in order (perhaps this week, we do a dry run or a single button press to prove concept). The user should see the effect on the app. This demonstrates the capability for general GUI automation via vision, moving beyond brittle pre-scripted coordinates.

Week 11: Cross-Device Orchestration & Mobile Extensions
- **Task**: Finalize the integration of mobile-specific actions and ensure the assistant works across the phone and laptop in harmony.
- **Key Actions**:
- Implement **Android device control** via ADB: Set up the Pixel for wireless ADB (developer options). Write scripts on the PC that can send ADB shell commands to the phone. For example, launching an app (`adb shell monkey -p com.app.package 1`) or simulating taps (`adb shell input tap x y`). Also use

ADB screencap to capture phone screen if needed and possibly run OmniParser on it (though Pixel's screen is smaller, OmniParser might still parse Android UI). Test a simple command: "Open YouTube on my phone" -> the PC sends ADB command to Pixel which opens YouTube.
- Implement **iOS Shortcuts** triggers: On the iPhone, create a few personal Siri Shortcuts for tasks like "Take a Photo and send to PC" or "Text my wife I'm driving" etc. Each can be exposed via a URL (using the `shortcuts://run-shortcut?name=XYZ&input=...` scheme or a web hook through a third-party like PushCut or iOS automation server). From the PC, invoking these URLs (if the phone is on same network) will prompt the shortcut on iOS. There's some inherent limitation (user might have to confirm on phone), but at least demonstrate one such integration.
- Synchronize **notifications**: If the assistant wants to alert the user and they are not at the PC, have it send a notification to the phone. Android and iOS apps can show notifications (Flutter can handle cross-platform notifications easily). Implement that: e.g. if a long-running task finishes on PC, the Pixel app receives a push (maybe use Firebase Cloud Messaging or since Tailscale exists, we could even send via our channel). For now, local network might not wake a phone if app is backgrounded; using a push service might be necessary but could be overkill in 12 weeks. A simpler approach: have the Pixel app stay connected (via WebSocket) and when something happens, it creates a local notification. The iOS app might not be allowed to in background, but we can show if app is foreground.
- Cross-device **context sharing**: ensure that if a conversation starts on one device, the context is accessible on another. For example, user asks on phone, "Remind me to buy milk tomorrow." Later on PC they ask, "What did I ask you to remind me?" – the assistant on PC should know about the reminder set via phone. This means the memory (Zep) and task list are shared. We probably have that by design since memory is centralized, but test explicitly.
- UI polish on mobile: make sure the mobile app (Flutter or native) has at least the ability to start/stop listening and display responses. It should feel like a remote client to the same brain.
- **Deliverable**: **Unified cross-device operation**. The user can seamlessly invoke Chintu on either desktop or mobile for various tasks. For instance, they might use voice on the phone to dictate an email, which Chintu drafts and then actually sends via the desktop (through an email client or webmail). Or set a phone alarm from the PC. This week's success criteria is a **fluid multi-device experience** – Chintu feels like one assistant living in all devices.

Week 12: Testing, Security Audit, and Launch Preparation
- **Task**: Finalize the project with thorough testing, security hardening, documentation, and fallback planning.
- **Key Actions**:
- Conduct a **full security review**. Analyze each component for vulnerabilities: ensure all inter-device calls are authenticated (e.g. only accept requests from known device IPs or with shared secret). Verify that sandboxing is effective – try intentionally malicious input (like "delete all files") and confirm the assistant does not execute it due to checks. Implement the "Constitution" safety prompts rigorously (e.g. policies preventing self-modification or dangerous acts) [60] . Add explicit user confirmation requirements for high-risk operations (the assistant should say, "Are you sure? [yes/no]").
- Add a **rollback mechanism** for self-improvement: if the agent does attempt to modify its code or add a plugin, ensure we have version control (git) in place and the ability to revert automatically if the new code fails tests [60] . This might include writing a test suite and running it whenever code is changed by the agent.
- **Performance tuning**: Profile the system. Identify slow points – maybe the first run of the LLM is slow due to loading (so keep it loaded), or memory queries could be batched. Fine-tune model parameters (maybe reduce LLM max tokens if not needed to speed up, or adjust prompt formatting to reduce overhead). Possibly incorporate **speculative decoding** fully if not done – e.g. use a 1B model to generate a rough draft to overlap with big model processing.

- **Resource check**: Ensure that the system can run continuously without hogging too much battery or CPU when idle. Implement an idle mode – e.g. if no interaction for a while, unload heavy models from RAM, but keep wake-word running minimally (especially on mobile to save battery). The Pixel's thermal limits for constant use should be respected (e.g. don't run Whisper continuously, only on trigger) [10] .
- Plan **fallback modes**: For example, if the local LLM is failing to answer certain queries accurately, decide when to route to cloud – perhaps based on a confidence score or if user explicitly says "use GPT-4". Implement a simple toggle or automatic trigger after one wrong attempt. Also, if one device is offline, the system should still work on the remaining ones (maybe with reduced functionality). Test by turning off the Mac and see if Dell alone handles it, turn off Dell and see if Mac can pick up (maybe not if model doesn't fit, but at least handle it gracefully).
- **User testing**: If possible, involve a friend or colleague to use the assistant for a day and provide feedback. Observe any usability issues, and fix obvious bugs.
- **Documentation**: Write comprehensive documentation for the system: architecture, how to set up each component, and user guide for features. This is crucial for maintainability and for any future expansion by the developer.
- **Deliverable**: A **production-ready v1.0** of Chintu. By the end of week 12, the assistant should be ready for its initial release. We will have a summary report of test results (e.g. "passed 50 scenario tests, identified 3 edge cases to fix in future"), and a clear deployment instruction for the solo developer to actually use Chintu day-to-day. Importantly, we will have a **contingency plan** documented: for each major risk identified below, an approach to mitigate or recover.

The above timeline is ambitious but structured to prioritize core capabilities first (a functional local assistant by week 5-6) and then layering on more advanced multimodal and agentic features. Each week's work builds towards the end goal of a robust, cross-device assistant.

Resource and Hardware Requirements

To implement and run Chintu effectively, certain hardware capabilities and resources are needed. Below we break down the role of each device in the distributed setup and the recommended specifications:

Device & OS	Role in Chintu Cluster	Recommended Specs / Resources
Dell Inspiron (Windows 10/11) – Primary Node	*Orchestrator & Heavy Compute*: Runs the main LLM inference, memory server, and vision processing. Also coordinates other devices. **Workload**: LLM model hosting (possibly 70B model sharded), OmniParser for GUI, etc.	**CPU/GPU**: Multi-core CPU (Intel i7/AMD Ryzen 7) and a dedicated GPU (e.g. NVIDIA RTX 3060 or better with 8GB+ VRAM) for accelerating vision models [61] . **Memory**: 32 GB RAM (to comfortably hold large models or multiple smaller ones) [61] . **Storage**: 1+ TB SSD (model files can be 30-60GB, plus Docker images). **Network**: Stable Wi-Fi (or Ethernet) for low-latency links to others.

Device & OS	Role in Chintu Cluster	Recommended Specs / Resources
MacBook Air (macOS) – Secondary Node	*Inference Partner*: Participates in distributed LLM inference and serves as backup orchestrator if needed. Can also handle less intensive tasks if primary is busy. **Workload**: Running parts of the model (through prima.cpp), possibly handling audio or personal Mac-specific automations.	**Chip**: Apple M1 or newer (with Neural Engine, though not used directly by our system). **Memory**: 16 GB unified memory (the more the better) [62] . **Storage**: 512 GB (for local data/cache). **Network**: Wi-Fi 6 for faster mesh performance. The Mac's efficiency helps it run tasks with low power draw when the Dell is off.
Google Pixel 7 Pro (Android) – Edge Node	*Mobile Sensor & Privacy Gateway*: Always-listening device for voice input (wake-word detection and initial STT). Performs first-layer privacy filtering on captured data. Also executes Android-specific tasks (calls, texting, etc.) upon instruction. **Workload**: Running a lightweight ASR (or feeding audio to PC), running Presidio/GLiNER for text filtering, and handling user notifications/output. Potentially holds the LLM's KV cache or smaller model layers to assist inference [10] .	**CPU/GPU**: Google Tensor G2 chip – adequate for small ML tasks (like wake-word, small NER). No additional GPU needed. **Memory**: 8 GB RAM – must manage it carefully to not overload (no huge models here). Keep background services minimal to avoid Android killing our process (disable battery optimizations). **Battery**: Will be under constant partial use; expect noticeable drain, so possibly keep device plugged in during heavy use or optimize by duty-cycling the microphone. **Thermals**: Pixel can heat up if doing prolonged ML; ensure tasks like Whisper transcription use as little time as possible or offload to PC after capturing audio [10] .
Apple iPhone XS (iOS) – Auxiliary Controller	*User Interface & iOS Automator*: Primarily serves as a client for user interaction on iOS and executes iOS-specific shortcuts/commands (since iOS is restrictive for background AI tasks). **Workload**: Capturing user voice or text input when on foreground, displaying responses, and triggering iOS Shortcuts for actions like sending messages or opening apps [63] . Limited or no direct LLM computation.	**CPU**: A12 Bionic – sufficient for UI and light tasks, but not used for heavy AI. **Memory**: 4 GB RAM – limit background operation (app might not run background long anyway). **Storage**: 64+ GB – only needed for app and caching some data. **Network**: Wi-Fi connectivity to the cluster (via Tailscale) is crucial. *Note*: Because the XS is older, battery and performance constraints mean it's basically a portal to the assistant, not a processor.

In addition to device specs, **general infrastructure** requirements include:
- **Software & Dependencies**: Windows and macOS should have Python 3.10+ and Docker installed. NVIDIA CUDA drivers on Windows for GPU use. Xcode and Android SDKs for building mobile apps. Tailscale installed on all devices.
- **Memory for Models**: The total memory across devices dictates the size of local LLM we can run. With 32GB on Dell + 16GB on Mac, we have ~48GB. A 70B quantized model might fit if 4-bit (~40GB) spread across them. If that's tight, we drop to a 33B model (~20GB) which is easily handled. Disk space for multiple

model versions and checkpoints should be ample.
- **Cloud Resources (Optional)**: API access to OpenAI and Google requires accounts and usage budgets. We estimate minimal usage, perhaps <$20/month if only occasionally calling GPT-4o for complex queries [64]. Ensure an internet connection for these calls when needed.
- **Misc Hardware**: A good quality microphone for the PC (for voice input when at desk), and possibly a webcam for gesture input. If gesture control is a priority, consider an *Intel RealSense* or Leap Motion device for more precise hand tracking, though a normal webcam can suffice with proper algorithms. For testing voice output, having speakers or a smart speaker to compare might help.
- **Power and Availability**: If Chintu is to be always-on, the main devices (Dell or Mac) should be running most of the time. Possibly set the Dell to not sleep or to wake on voice if possible. The mobile devices need to be plugged in or at least charged frequently if used continuously (especially Pixel as hotword device).

Overall, this setup is achievable with a high-end consumer laptop and phone – which matches the solo developer's hardware list. The **Dell with an NVIDIA GPU** is strongly recommended to unlock the full vision and heavy-model potential [61]. Without a GPU, the vision processing (OmniParser, large LLM inference) might be too slow; in that case, one might opt to use the cloud for vision (e.g. call an API for screen understanding) as a fallback. The plan above assumes the developer can invest in or already has a decent GPU – given the target is cutting-edge personal AI, this is justified.

Development and Security Best Practices

Creating a powerful AI assistant that runs locally across devices introduces significant complexity, so adhering to solid development and security practices is crucial from day one. Below we outline the key practices:

- **Modular Code Design**: Follow a microservice-like architecture – each major component (LLM service, memory service, UI client, etc.) should be decoupled and communicate via clear interfaces. This allows independent development and testing of components. For example, the voice input module can be tested with a stubbed backend, and the LLM module can be tested with preset prompts. Use a **plugin architecture** for tools so that adding a new tool (e.g. calendar integration) doesn't require altering core logic significantly.

- **Version Control and CI/CD**: Use Git for version control rigorously. This is especially important as the project spans multiple environments (some code runs on PC, some on mobile). Set up continuous integration tests if possible – at least running the core Python test suite on pushes. Given we have AI in the loop, also use a **two-tier testing approach**: basic unit tests for deterministic parts (e.g. memory DB operations, parsing functions) and integration tests that may involve the LLM (possibly with a fixed random seed or using a stub model). The OpenHands SDK paper highlights a similar strategy of combining mocked tests with real model tests for agents [65] [66] – we should do likewise, e.g., daily run a test conversation end-to-end to catch regressions.

- **Secure Coding Practices**: Treat any code that interfaces with system commands or external input as potential vulnerability points. Validate all inputs (even though it's a personal assistant, it might process external data from web or user files). Use parameterized queries or ORMs for any database access to avoid injection (with Zep this is likely handled). When executing shell commands (via tools), **escape or whitelist** allowed commands. E.g., if we allow `run_code`, it should run in a sandbox

where even if malicious code is generated, it can't harm the actual system [67] [68]. The sandbox (Docker container) has restricted permissions (no root access to host, limited network if any, etc.).

- **Least Privilege**: Run services with minimal permissions. For instance, the Windows agent process doesn't need admin rights for most things – running as user reduces risk. The Android app should request only absolutely necessary permissions (Microphone, and maybe Accessibility if we go that route for UI control; avoid requesting contacts, location unless a feature demands it). If using Apple Script or Shortcuts on Mac, consider the new **Privacy Permissions** – the app will need Accessibility permission to control UI, which user must grant. Document these needs clearly so the user (developer) knows and consents.

- **Encryption and Data Protection**: All communication between devices is encrypted by default through Tailscale (WireGuard). Even so, we may add application-layer encryption for sensitive payloads as an extra layer (especially if any data traverses through a cloud server – but in our case, it's direct peer-to-peer). Implement **end-to-end encryption** for particularly sensitive sync, e.g., if we ever sync a secure note between devices, we could encrypt it with a user key before storing in memory [69]. On disk, the memory database or any logs should ideally be encrypted at rest or at least stored in the user's secure locations (like Keychain or Android's internal storage).

- **API Key Management**: Any cloud API keys (OpenAI, etc.) should not be hardcoded. Use environment variables or a secure vault. The keys should only be loaded into memory when needed. Provide a config file mechanism where the user (dev) places their keys, and ensure that file is in .gitignore (not checked into version control accidentally). Rotate keys if suspect any leak.

- **Privacy Measures**: Besides the PII filtering, maintain a **privacy log** – basically record whenever data might leave the local environment. For example, if a user query triggers a cloud API call, log what was sent (with masks) and to whom, so the developer can audit that no private info was sent upstream. Also, as a rule, default to not using cloud unless explicitly needed. This keeps with zero-knowledge ideals [70]. If possible, implement a setting where even if the user tries to do something that normally calls cloud (like a very complex task), the assistant asks "This may use a cloud AI, do you allow?" each time or per session.

- **RASP and Monitoring**: Employ **Runtime Application Self-Protection** techniques [69]. This includes monitoring the assistant's own actions at runtime to catch anomalies. For example, if the assistant's tool subsystem unexpectedly tries to access a file it never should (like a system DLL or something), flag that. We can build an internal policy like a mini-intrusion detection for the agent. Logging all actions with timestamp and outcome is valuable not just for debugging but for security review. If something goes wrong, these logs help pinpoint where.

- **Regular Security Audits**: Every few weeks (and definitely at week 12 before launch), do a structured audit – review code for any use of dangerous functions, ensure all subprocess calls either use safe APIs or are sandboxed. Use tools like linters or security scanners (Bandit for Python, etc.) to automate some checks. If the project becomes open-source or even if not, treating it with an enterprise security mindset is wise given it has access to personal data and system controls.

- **Fail-safe Mechanisms**: Build **fallbacks** for critical failures. If the LLM agent ever goes off-script or takes too long, have a watchdog that kills or resets it. If a tool execution returns an error or hangs,

have timeouts. The system should remain responsive and not require a full reboot if one part crashes – e.g., use multiprocessing such that the voice interface is one process, the LLM another, so if LLM process crashes, the voice interface can say "Sorry, something went wrong, restarting my brain" and attempt to relaunch it. Also, if one device fails, the others should not be stuck waiting indefinitely – implement timeouts for cross-device calls too.

- **User Consent and Control**: Provide the user (the developer in this case) clear control over Chintu's operations. For any irreversible or sensitive action (deleting files, sending messages, spending money, etc.), *always confirm with the user*. It's better to err on the side of caution: e.g. the assistant can draft an email, but user hits send. Or assistant can suggest "Shall I apply to this job [Company]?" and user confirms before it auto-submits. This not only prevents accidents but also builds trust in the system's actions.

By following these practices, we reduce the chance of catastrophic mistakes (like data leaks or system damage) and improve the maintainability of the project. Security is an ongoing process – as Chintu gains abilities, new potential abuses arise, so we will continuously update the safety rules and perform tests (like using known prompt injection attacks on it to see if it bypasses filters, and then patching those holes).

Cross-Device Integration Strategy

Integrating across four different devices and operating systems is a core challenge for Chintu. Our strategy is to use a combination of networked coordination and platform-specific adaptations to create a unified experience:

- **Mesh Networking & Coordination**: As described, **Tailscale VPN** links the devices so they act like they're on a local network. One device (the Dell) will typically host the **"brain" services** (LLM, memory, orchestrator). The other devices run lightweight **agent clients** that interface with the user and execute delegated tasks. The communication is done through **API calls or RPC** over the mesh. For instance: when you speak to the Pixel, the Pixel app sends the audio or transcript to the Dell's API `/processVoice`. The Dell computes response and decides an action. If action is "do something on Pixel", the Dell calls Pixel's API (or sends a push/WS message) with the command. This hub-and-spoke makes Dell a central coordinator. However, to avoid a single point of failure, the MacBook can also run a mirror of the orchestrator – possibly in standby – that can take over if Dell is offline. In practice, for v1, we'll likely rely on the primary node being on for full functionality. The architecture is flexible though: if the user is away from the PC, the Pixel alone still can handle many tasks (it has a small local model, etc., though not as powerful). We ensure that any critical data (like memory DB) is either hosted on a always-available device or periodically synced so that secondary can assume control if needed.

- **State Synchronization**: To present one unified persona, all devices share the assistant's state (context and memory). The **long-term memory (Zep)** is a shared knowledge base accessible to all nodes (Dell might host it, and Pixel/Mac query it over the network). Session state (ongoing conversation) can be synchronized by sending updates: e.g., if you talk to the Pixel and then continue on the Mac, the Mac's UI should display the last exchanges too. We can achieve this by storing recent conversation history in the memory store or a lightweight central cache, and when a client app connects it pulls the latest log. We might also send real-time events: when Pixel finishes processing a query, it can notify other UIs to update. Utilizing a **publish-subscribe** model (could be

as simple as a MQTT broker on the network or just WebSocket broadcasts from the orchestrator) will help keep multiple UIs in sync.

- **Unified User Interface Design**: With Flutter as our likely UI framework, we can maintain a consistent look and feel across desktop and mobile. The UI would consist of: a conversation view (chat bubbles), a waveform or indicator for listening, and maybe quick action buttons (like a microphone toggle, or a keyboard icon to switch input mode). We will design it responsive so that on a desktop it might appear as a sidebar or floating widget, and on mobile it's full-screen or a chat-style app. The **floating overlay** mode on desktop is key – it allows Chintu to sit atop other windows unobtrusively (like a little assistant head). Achieving that means handling window transparency and click-through (which Flutter on Windows/Linux can do, macOS needs a native plugin) [45]. On mobile, integration into OS (like appearing over other apps) is limited by OS policies, but Android can do a persistent notification or chat head, and iOS cannot (so just an app). Over time, integration might mean making it a voice-first interface (similar to how Siri/Alexa appear).

- **Device-Specific Integrations**: Each OS has unique integration points:

- *Windows*: Use PowerToys/Voice Access APIs or Win32 for some control, register global hotkey or wake-word service if possible. Possibly integrate with Windows Timeline or calendar for those tasks.
- *macOS*: Use AppleScript or Automation for deep control (opening apps, controlling settings), via the `osascript` CLI or newer Automation APIs. Accessibility API for reading screen contents (if OmniParser wasn't used, voiceover API could be an alt, but we prefer OmniParser).
- *Android*: Leverage Android's Intent system. We can craft specific intents to open apps or perform actions (if the Pixel is rooted or using ADB, we bypass needing official APIs). Also can use the Notification Listener API to let Chintu read notifications (with permission), so it could e.g. read aloud an incoming message if user asks. Over time, an Accessibility Service could allow the assistant to navigate the Android UI (scroll, click), but that requires careful handling and user trust. For now, ADB and direct intents are enough, albeit require developer mode.

- *iOS*: Very restricted, but Shortcuts automation is our friend. We will rely on Siri Shortcuts as an integration path. For example, a shortcut that, when run, takes a command from a server (which we set via a URL) and executes it. Without jailbreak or macOS-side assistance, iOS can't be fully controlled, so our strategy is that the iPhone is more of an output device and secondary input. If the user gives a command that involves iPhone (like "take a photo on my iPhone"), we might either prompt the user to do it manually or queue it until the user opens the app (then the app can call the camera). Documentation will clarify these limitations.

- **Tool and Data Sharing**: We use a single memory DB and possibly a shared filesystem or cloud drive for certain data exchange. For example, if Chintu downloads a PDF on PC for analysis but user wants to see it on phone, we could save it to a synced folder (like iCloud/Drive, or simpler, the PC could send file bytes via our API to the phone app which then offers to open or save it). Not everything needs full sync, but specific features like the personal knowledge store might be exported to all devices (in a read-only way) – e.g., a cached summary of user's key facts stored on phone for quick offline answers.

- **Consistent Identity and Settings**: All devices refer to the assistant as "Chintu" and use the same underlying persona configuration. User preferences (like voice gender for TTS, or wake-word on/off,

or default behavior settings) are stored centrally (perhaps in memory DB or a config file) and propagated. This ensures you don't have to configure each device separately. A small settings sync mechanism will be built – possibly just load from a common config on start.

- **Testing Integration**: During development, each new cross-device feature will be tested in incremental steps (as seen in timeline). We'll simulate network loss, device offline scenarios. We'll also ensure that if two inputs come at once (say user triggers voice on phone while also typing on PC), how do we handle it? Likely we'll queue or prioritize one (or ignore one with a polite message). Some locking might be needed to avoid race conditions (e.g. not running two LLM queries concurrently that conflict). We could allow concurrent different tasks if they don't conflict (like one device doing a web search while another doing something else) – future improvement, but not needed at start.

In summary, the integration strategy is to **centralize intelligence but decentralize execution**. The personal cluster behaves like a single AI presence distributed in hardware: whichever device the user interacts with, they access the same "mind" of Chintu. We use standard protocols and VPN to blur the lines between OS boundaries, and platform-specific hooks to give Chintu reach into each device's capabilities. This approach, while complex, is feasible with today's technology and ensures we meet the goal of a *distributed, privacy-preserving assistant* that leverages each device for what it does best.

Risk Assessment and Mitigation Plans

Despite careful planning, a project of this scope has inherent risks. Below, we enumerate major risk factors and outline mitigation and fallback plans for each:

1. Technical Feasibility Risks

- *Distributed LLM performance may fall short*: Running a large model across devices (prima.cpp) might have unanticipated latency issues or bugs. **Mitigation**: Start with smaller models to ensure functionality. If the 70B model is too slow or unstable, fall back to a 30B or 13B model which can run on one machine reliably. The system is designed to be model-agnostic, so we can trade off some capability for responsiveness. As a contingency, allow switching to cloud LLM for queries that need high intelligence that a smaller local model can't provide (with user permission).
- *Multi-modal integration complexity*: Voice, vision, and gesture all in one system can introduce a lot of complexity (e.g., audio processing and image processing might conflict for resources). **Mitigation**: Modularize and enable features one at a time (as per timeline). If one modality is causing trouble (say gesture control proving too inaccurate), we will deprioritize or disable it for the initial release and focus on the more critical voice and vision. Gesture control can be marked as experimental or left for a future update if not ready – it is not a showstopper feature.
- *Hardware limitations*: The iPhone XS and Pixel 7 might not handle background tasks as expected (iOS might suspend our app, Android might kill a background service that's heavy). **Mitigation**: Use platform-recommended methods (Foreground service on Android to keep alive; on iOS, accept the limitation and don't rely on it for always-on tasks). Plan fallback where if iPhone app is not running, the Pixel or other devices cover that functionality. If the Pixel is overheating or battery draining due to constant hotword detection, implement a quick toggle to turn off continuous listening and only listen on-demand. The user can then choose battery vs convenience.
- *Integration bugs*: The interaction between components might lead to race conditions or deadlocks (e.g., waiting on each other's response). **Mitigation**: Extensive testing, use timeouts for network calls. Implement

a watchdog thread that can detect if a cycle is stuck (no response in X seconds) and attempt to recover (maybe by restarting a component or at least logging the issue). Keep a manual override (maybe a keyboard interrupt or voice command "reset") that the user can use if it hangs, which restarts the agent process.

2. Privacy and Security Risks

- *Data leak to cloud*: Despite precautions, there's a risk that some private data could slip into a query to the cloud LLM (maybe the user explicitly asks it to summarize a private document and we send that to GPT-4o without thinking). **Mitigation**: Implement strong checks before any cloud call – if a query context contains anything marked sensitive by our filters, either scrub it or refuse cloud usage. The default will be local processing; cloud is opt-in per query and will carry a warning ("This will send data to external API"). Additionally, monitor the outputs from cloud (though encrypted in transit, we will trust OpenAI/GCP's privacy terms, but ensure we don't accidentally send more than needed).
- *Malicious use or runaway agent*: If someone compromised the assistant or prompt-injected it cleverly, it might try to do harmful actions (like delete files, or send unwanted messages). **Mitigation**: Multi-layered: the Constitution prompt explicitly forbids destructive actions without user approval [60]. The agent state machine will gate actions so that anything sensitive requires a confirmation state. Also, we physically sandbox execution of any code and limit file writes. If the assistant tries to step out of bounds, those actions either won't execute (no permission) or the user will be alerted. We'll log all such attempts. As a fallback, a *panic button* approach: an easy way to fully stop all agent actions (maybe a voice command "Abort" or a keyboard shortcut that shuts down the orchestrator) in case it's doing something unexpected.
- *Security vulnerabilities*: The assistant might unintentionally create a security hole (e.g., opening a port on the network or executing a command that exposes something). **Mitigation**: Keep all services behind the VPN (so not exposed to the internet at large). Regularly update dependencies to patch known issues. Use a firewall to restrict inbound/outbound communications to only necessary addresses (maybe limit cloud API domains, etc.). Run static analysis tools on our code to catch risky patterns. If a vulnerability is discovered (say the RPC server on PC could be accessed by someone if they got into VPN), consider adding an authentication or at least not using default ports. Since this is personal, risk is lower than a public service, but we treat it seriously because the devices have sensitive info.

3. Timeline and Project Management Risks

- *Solo developer constraints*: It's a lot for one person in 12 weeks. Illness, unforeseen difficulties, or simply underestimation could derail timeline. **Mitigation**: The plan is already phased by priority. If behind schedule, focus on delivering a solid core (text and voice on desktop with some automation) rather than half-working everything. The timeline is flexible: e.g., gesture control (nice-to-have) can slip out to a later release if needed. We also identified parallelizable tasks (some work on voice vs memory vs UI). As a solo dev, you can alternate tasks to avoid burnout – e.g., if blocked by a model issue one day, spend time on UI design. The key is constant reevaluation each week and adjusting scope. This blueprint itself will help to keep track of what's essential.
- *Integration hell at the end*: Many projects face a crunch when all pieces come together and don't immediately gel. We allocated Week 8 and 12 specifically for integration testing and bugfix to reduce this risk. Also adopting continuous integration (testing combined flows every few days, not waiting till the end) is how we mitigate it. If by week 10 things are shaky, we might skip some Week 11 features to focus on stability. It's better to have a reliable assistant with 90% features than a glitchy one with 100%.

4. User Adoption and Experience Risks

- *Accuracy and usefulness*: If the local models are not as accurate or the assistant fails frequently (mishears,

gives wrong info), the user might get frustrated and not trust it. **Mitigation**: Curate the prompt and tune the system to avoid known pitfalls. E.g., incorporate few-shot examples in prompts to improve the assistant's reliability on tasks like job applications or coding style. Use the memory to personalize (if it knows the user's context, it will naturally do better). For factual queries, consider double-checking with a web tool or using cloud for a second opinion if local answer seems uncertain. Communicate to the user when the assistant is unsure rather than confidently wrong – e.g., say "I'm not entirely sure, would you like me to search the web?" This transparency can turn potential failures into collaborative interactions.
- *Latency or UX issues causing abandonment*: If using Chintu feels laggy or cumbersome (too many confirmations or long waits), the user might not incorporate it into daily routine. **Mitigation**: Identify high-frequency tasks and optimize for them. E.g., set timers, opening apps, quick Q&A should be snappy. Use techniques like wake-word to reduce friction in activation. At the same time, keep confirmations only for destructive things; don't ask "are you sure?" for every small action or it ruins UX. Where safe, just do it and perhaps allow an "undo" (for example, if it sends an email, offer an undo option within a few seconds). Gathering feedback is key – since the user is the developer, they can adjust things that annoy them on the fly. It's an advantage here that the user and developer are the same; we can iteratively refine the UX to personal preference.

5. Maintenance and Future-proofing Risks
- *Dependency changes*: AI moves fast; new versions of tools (or deprecations) can break our system (e.g. an API we use might change, or an open-source project might stop being maintained). **Mitigation**: Containerize and pin versions of critical components so that the system is reproducible. E.g., even if Zep moves on, we have a Docker image of the version we used. Track AI news – perhaps join communities or mailing lists of these projects – so we get heads-up on changes. The blueprint also chose mostly open or self-hostable tools to reduce external reliance; that helps if companies change their offerings (we won't suddenly lose a cloud service, for instance).
- *Scalability for future tasks*: Today it's one user on four devices; tomorrow maybe you add another device or a family member wants to use it. The architecture might need scaling. If we didn't consider multi-user, there might be assumptions that break. **Mitigation**: While multi-user is out of scope, designing the memory and agent with user IDs (even if one) can make extension easier. And the distributed design could allow adding more nodes (like another laptop) relatively easily. We keep that in mind when writing code (no hard-coded device names, etc., use discovery).

Fallback Plans:
In the worst case that some parts just don't work by delivery, we ensure the core functionality doesn't suffer. For example, if distributed inference is too unreliable, fallback to *single-device inference* (run everything on the Dell, which likely can handle a mid-sized model alone). If the vision module isn't ready, rely on simpler automation via known element IDs or postpone advanced GUI automation – the assistant can still do plenty via API and command-line without visually understanding everything. If voice always-listening is problematic (maybe false triggers), a fallback is push-to-talk activation (e.g. double-press a volume button on phone to activate). We will document these alternatives so that the user can still use Chintu effectively even if some cutting-edge aspects aren't 100% yet.

In summary, by anticipating these risks and planning mitigations, we aim to deliver a robust, secure, and useful assistant. The motto is *"plan for the best, prepare for the worst."* We leverage the fact that this is an in-house personal project: we can make pragmatic choices and cut features if needed for stability, without external customer impact. Over time, as each risk is addressed and the system solidifies, Chintu can

gradually approach that ambitious vision of a truly smart, cross-device personal AI that one can rely on daily [71] [72].

---

[1] [4] [5] [6] [8] [9] [10] [11] [12] [13] [14] [15] [16] [17] [18] [31] [32] [33] [34] [38] [39] [47] [48] [49] [52] [54] [55] [56] [60] [61] [62] [63] [64] [70] [71] [72] AI Assistant Architecture and Development Plan.pdf
file://file_00000000d47471f5b03dfd89f665ca08

[2] [3] [46] Prima.cpp: Fast 30-70B LLM Inference on Heterogeneous and Low-Resource Home Clusters | OpenReview
https://openreview.net/forum?id=h0LjpOG1jq

[7] GLiNER: The Lightweight Alternative to LLMs for Custom NER | CodeCut
https://codecut.ai/gliner-the-lightweight-alternative-to-llms-for-custom-ner/

[19] Daniel Bourke - machinelearning #ai #opensource - LinkedIn
https://www.linkedin.com/posts/mrdbourke_machinelearning-ai-opensource-activity-7259443923683614720-mh6s

[20] [21] [40] [41] [42] [43] [44] [45] AI Assistant UI_UX Development Plan.pdf
file://file_00000000dfcc71f5a6ba024fcbab2353

[22] [23] [24] OmniParser V2: Turning Any LLM into a Computer Use Agent - Microsoft Research
https://www.microsoft.com/en-us/research/articles/omniparser-v2-turning-any-llm-into-a-computer-use-agent/

[25] [26] [57] [58] [59] [65] [66] [67] [68] The OpenHands Software Agent SDK: A Composable and Extensible Foundation for Production Agents
https://arxiv.org/html/2511.03690v1

[27] [28] [29] [30] [69] Chintu.pdf
file://file_00000000e4d471f58597dc98a9ba2c7e

[35] GPT-4o - Wikipedia
https://en.wikipedia.org/wiki/GPT-4o

[36] Gemini 2.5: Our newest Gemini model with thinking
https://blog.google/technology/google-deepmind/gemini-model-thinking-updates-march-2025/

[37] [53] ollama/ollama: Get up and running with OpenAI gpt-oss ... - GitHub
https://github.com/ollama/ollama

[50] Docker + E2B: Building the Future of Trusted AI
https://www.docker.com/blog/docker-e2b-building-the-future-of-trusted-ai/

[51] The New Era of Cloud Agent Infrastructure: In-Depth Analysis of E2B ...
https://jimmysong.io/blog/e2b-browserbase-report/